

Problema de Roteamento de Veículos Multiobjetivo com Coleta Seletiva

Luciana Pereira de Assis*, André Luiz Maravilha Silva,
Felipe Campelo Franca Pinto, Alessandro Vivas Andrade e Jaime Arturo Ramírez

Resumo: Este capítulo apresenta uma abordagem multiobjetivo para o problema de roteamento de veículos com coleta seletiva, cujos objetivos são a minimização dos custos das rotas e das demandas de coletas não atendidas. Propõe-se uma estrutura de dados que melhor se ajusta ao problema, estruturas de vizinhanças que exploram ambos objetivos do problema e um algoritmo que verifica a viabilidade de uma solução com menor custo computacional. Para solucionar o problema, são discutidas três meta-heurísticas: o ILS multiobjetivo (MOILS), NSGA-II e o método ϵ -Restrito, as quais são aplicadas à 14 instâncias contendo entre 50 e 199 consumidores. Os resultados indicam que o MOILS é superior aos outros algoritmos, obtendo valores médios para cobertura, hipervolume e cardinalidade significativamente melhores.

Palavras-chave: Problema de roteamento de veículos, Meta-heurística, Otimização multiobjetivo.

Abstract: *This chapter presents a multiobjective approach for the vehicle routing problem with optional collections, whose objectives are the minimization of the route costs and of the not fulfilled collection demands. It proposes a data structure that best suits the problem, neighborhood structures that exploit both goals of the problem and an algorithm that checks the feasibility of a solution with lower computational cost. To solve the problem, three metaheuristics are discussed: the multiobjective iterated local search (MOILS), NSGA-II and the ϵ -Constrained method, which are applied to fourteen instances containing between 50 and 199 customers. The results indicate that the MOILS outperforms the other approaches, obtaining significantly better average values for coverage, hypervolume and cardinality over the set of used test problems.*

Keywords: *Vehicle Routing problem, Metaheuristics, Multiobjective optimization.*

1. Introdução

O problema de roteamento de veículos (VRP, *Vehicle Routing Problem*) é um dos mais importantes e mais amplamente estudados no contexto da otimização combinatória, por englobar grande parte dos problemas reais de logística de transporte. Em geral, este problema consiste em definir rotas otimizadas que atendam às demandas de um conjunto de consumidores considerando uma frota finita de veículos capacitados.

Uma vez que existe um custo de transporte associado entre cada par de consumidores, e entre cada consumidor e o depósito, o objetivo do problema é definir rotas de menor custo total. Este problema é caracterizado por uma natureza combinatória, ou seja, o domínio da função a ser otimizada é dada pela permutação ideal de um conjunto de arestas que conectam os pares de consumidores e conectam os consumidores ao depósito. Com o aumento no número de consumidores, torna-se extremamente difícil encontrar soluções ótimas a partir de métodos exatos.

Assim sendo, diversas meta-heurísticas têm sido propostas para solucionar problemas para os quais métodos exatos não são capazes de fornecer soluções em um tempo computacional viável. Dentre estas, destacam-se os métodos baseados em buscas locais, como a Busca Local Iterativa (ILS, *Iterated Local Search*) (Lourenço et al., 2003), e os métodos baseados em população como os Algoritmos Genéticos (Holland, 1992).

Em alguns problemas de roteamento encontrados na literatura as demandas dos consumidores podem ser tanto de entrega quanto de coleta de mercadorias. O estudo deste fluxo de material está inserido no contexto da logística reversa, que planeja toda a atividade de entrega dos produtos e, também, o caminho inverso, ou

* Autor para contato: lpassis@ufvjm.edu.br

seja, o retorno dos produtos à central de distribuição. Algumas indústrias estão cada vez mais preocupadas com o gerenciamento do retorno do material consumido para o depósito a fim de reciclá-los, remanufaturá-los ou agregar novos valores aos mesmos.

O problema de roteamento de veículos com coletas e entregas simultânea (VRPSPD, *Vehicle Routing Problem with Simultaneous Pickup and Delivery*) é um problema básico de logística reversa. O objetivo deste problema é minimizar os custos de transporte, atendendo a todas as demandas de coleta e entrega simultaneamente (Subramanian et al., 2011).

O problema de roteamento de veículos com serviço de coleta e entrega, proposto por Sural & Bookbinder (2003), ilustra situações onde o atendimento das demandas de coleta não é obrigatório. Este problema, denominado *Single-Vehicle Routing Problem with Unrestricted Backhauls*, associa um lucro a cada demanda de coleta, caso esta seja atendida. O objetivo consiste em definir um percurso que atenda a todas as demandas de entrega e, também, realize as coletas que forem consideradas lucrativas, minimizando uma função objetivo denominada custo líquido (custo total da rota subtraído do lucro total obtido através das coletas realizadas). Um algoritmo exato (*Branch-and-Bound*) é aplicado para solucionar instâncias do problema com poucos consumidores. Gutiérrez-Jarpa et al. (2010) apresentam uma abordagem exata (*Branch-and-Price*) para solucionar problemas de roteamento de veículos com entrega e coleta seletiva e janela de tempo. Outro problema similar é apresentado por Gribkovskaia et al. (2007), denominado *Single Vehicle Routing Problem With Delivery and Selective Pickups*. Neste problema, os consumidores podem ser visitados duas vezes; uma vez para realização das entregas e outra para coletas. O objetivo do problema também é a minimização do custo líquido.

Apesar da natureza multiobjetivo destes problemas, os trabalhos citados apresentam uma modelagem mono-objetivo e fazem uso de um lucro associado às demandas para diferenciá-las. Contudo, o estudo de problemas de roteamento de veículos com múltiplos objetivos vem apresentando maior destaque nos últimos anos (Jozefowicz et al., 2008), mostrando que em algumas situações reais é necessário visualizar diversos cenários, dados os diversos objetivos do problema, para se tomar uma decisão. Assim sendo, ao modelar problemas de maneira multiobjetivo, estes se tornam mais interessantes para aplicações práticas.

Este capítulo aborda o problema de roteamento de veículos multiobjetivo com coleta seletiva (MOVRPSP, *Multiobjective Vehicle Routing Problem with Selective Pickup*) cujos objetivos são: minimização dos custos de transporte e minimização das demandas de coletas não atendidas. Esta formulação pode ser vista como uma generalização do VRPSPD, uma vez que a restrição de atendimento de todas as demandas de coleta se transforma em um objetivo do problema.

Para avaliar diferentes estratégias de resolução do MOVRPSP, este trabalho apresenta três algoritmos: Busca Local Iterativa Multiobjetivo (MOILS, *Multiobjective Iterated Local Search*) (Geiger, 2006), Algoritmo Genético de Ordenação Não-Dominante II (NSGA-II, *Non-Dominated Sorting Genetic Algorithm-II*) (Deb et al., 2002) e heurística P_ϵ -ILS, baseada no método ϵ -Restrito (Chankong & Haimes, 1983). Estes algoritmos exploram as características dos algoritmos baseados em busca local, em população e nos métodos tradicionais de resolução de problemas multiobjetivo, respectivamente.

Este capítulo está organizado como se segue. A Seção 2 apresenta a definição do problema e sua modelagem matemática multiobjetivo. Em seguida, na Seção 3, as estruturas de dados e os algoritmos aplicados na resolução do problema são apresentados. Os resultados computacionais são mostrados na Seção 4 e, por fim, na Seção 5 são feitas as conclusões deste trabalho.

2. Definição do Problema

O problema de roteamento de veículos multiobjetivo com coleta seletiva (MOVRPSP) consiste em definir um conjunto de rotas que atenda a todas as demandas de entrega, minimizando os custos de transporte e o número de coletas não realizadas. Cada consumidor deve ser visitado uma única vez e o atendimento parcial das demandas não é permitido.

Os dois objetivos a serem otimizados são conflitantes, ou seja, melhorias em um provocam degradações no outro, não existindo uma solução que seja ótima para ambos os objetivos simultaneamente. Assim, não é uma tarefa trivial identificar se uma determinada solução é de melhor qualidade em relação a outra. Isto leva ao conceito de dominância de Pareto (Definição 1), sendo a solução do problema um conjunto de soluções não dominadas. Neste trabalho existe uma diferença entre um conjunto de soluções não dominadas e um conjunto Pareto-ótimo. Um conjunto de soluções não dominadas é definido no contexto de uma amostra do espaço de busca, enquanto que o conjunto ótimo de Pareto é definido em relação a todo o espaço de busca (Definições 2 e 3).

Definição 1 (Dominância de Pareto) Dado um vetor objetivo $Z(x) = (f_1(x), \dots, f_m(x))$, com m funções objetivo para minimização. Uma solução x' domina uma outra solução x'' se, e somente se, para todos os

objetivos $f_i(x') \leq f_i(x'')$ com $i = 1, \dots, m$ e, existe pelo menos um objetivo $f_i(x') < f_i(x'')$. Esta relação de dominância é representada pela notação $x' \prec x''$ (Coello Coello et al., 2007).

Definição 2 (Solução Pareto-Ótima) x^* é uma solução Pareto-Ótima se não existe qualquer outra solução x tal que $x \prec x^*$, ou seja, x^* não é dominada por nenhuma outra solução (Coello Coello et al., 2007).

Definição 3 (Conjunto Pareto-Ótimo) O conjunto $\mathcal{X}^*$ é um conjunto Pareto-Ótimo se é composto por todas as soluções Pareto-Ótimas. O conjunto-imagem $\mathcal{Y}^*$ associado ao conjunto Pareto-Ótimo é denominado fronteira Pareto-Ótima (Coello Coello et al., 2007).

2.1 Modelagem matemática

A modelagem matemática apresentada nesta seção é uma adaptação da modelagem proposta por Montané & Galvão (2006) para o VRPSPD.

Seja $V = \{1 \dots n\}$ o conjunto de consumidores e V_0 o conjunto de consumidores mais o depósito ($V_0 = V \cup \{0\}$). Um custo $c_{ij} \geq 0$ é associado a cada arco que conecta os pares de consumidores e os consumidores ao depósito. As demandas de coleta e entrega do consumidor j são representadas por p_j e d_j , respectivamente. O número de veículos disponíveis é dado por $\bar{k}$, sendo Q a capacidade destes. O parâmetro y_{ij} é o somatório das cargas coletadas entre o depósito e o nó i (inclusive) dirigida ao nó $j \in V_0$. Já z_{ij} é o somatório das demandas de entrega dos consumidores visitados após o nó i (exclusive) e dirigida ao nó $j \in V_0$. A variável de decisão binária x_{ij}^k assume valor 1 se a aresta (i, j) é transposta pelo veículo k e 0, caso contrário. A variável de decisão binária ℓ_j recebe valor unitário se a demanda de coleta do consumidor j é satisfeita e 0, caso contrário.

O modelo matemático pode ser expresso por:

$$\text{Minimize } \begin{cases} \sum_{k=1}^{\bar{k}} \sum_{i=0}^n \sum_{j=0}^n c_{ij} x_{ij}^k \\ \sum_{j=1}^n p_j (1 - \ell_j) \end{cases} \quad (1)$$

Sujeito a:

$$\sum_{i=0}^n \sum_{k=1}^{\bar{k}} x_{ij}^k = 1, \quad j = 1, \dots, n \quad (2)$$

$$\sum_{i=0}^n x_{ij}^k - \sum_{i=0}^n x_{ji}^k = 0, \quad j = 0, \dots, n \text{ e } k = 0, \dots, \bar{k} \quad (3)$$

$$\sum_{j=1}^n x_{0j}^k \leq 1, \quad k = 1, \dots, \bar{k} \quad (4)$$

$$\sum_{i=0}^n z_{ij} - \sum_{i=0}^n z_{ji} = d_j, \quad \forall j \neq 0 \quad (5)$$

$$\sum_{i=0}^n y_{ij} - \sum_{i=0}^n y_{ji} = p_j \ell_j, \quad \forall j \neq 0 \quad (6)$$

$$y_{ij} + z_{ij} \leq Q \sum_{k=1}^{\bar{k}} x_{ij}^k, \quad i, j = 0, \dots, n \quad (7)$$

$$x_{ij}, \ell_j \in \{0, 1\}, \quad i, j = 0, \dots, n \quad (8)$$

$$y_{ij}, z_{ij} \geq 0, \quad i, j = 0, \dots, n \quad (9)$$

As duas funções objetivo, dadas na Equação (1), representam o custo total das rotas e as coletas não realizadas, ambas devem ser minimizadas.

Estas funções objetivo devem satisfazer algumas restrições. Cada ponto de demanda deve ser visitado por um único veículo (Equação (2)). A Equação (3) representa a restrição de conservação do fluxo. A Equação (4) indica que $\bar{k}$ veículos, no máximo, podem ser utilizados. A Equação (5) obriga a satisfação de todas as demandas de entrega. A Equação (6) garante o atendimento da demanda de coleta do consumidor j quando a variável de decisão ℓ_j assumir valor unitário. A Equação (7) define que as demandas devem ser transportadas

nos arcos incluídos na solução e ainda impõem um limite para a carga total transportada pelo veículo. A Equação (8) representa a restrição de integralidade e, finalmente, a Equação (9) representa as restrições de não-negatividade para demandas de coleta e entrega.

3. Estratégias de Solução

Nesta seção serão apresentadas as ferramentas e métodos utilizados para resolução de problemas de roteamento de veículos multiobjetivo com coleta seletiva.

3.1 Estrutura de dados

A maneira como os dados de entrada de um problema são armazenados interferem diretamente na eficiência dos algoritmos implementados. Os problemas de roteamento de veículos podem ser representados através de um grafo completo, onde os consumidores e o depósito compõem o conjunto de vértices e as arestas representam as conexões entre consumidores e entre consumidores e depósito. Existem diversas formas de representar um grafo e este capítulo apresenta uma nova estrutura a fim de unir as vantagens de duas estruturas bem tradicionais: matriz de adjacência e lista de adjacência.

Para a nova estrutura, foi implementada uma matriz de adjacência, onde cada elemento (i, j) da matriz é uma célula de uma lista duplamente encadeada composta por: vértice adjacente a i , o custo da aresta (i, j) e dois ponteiros. Partindo da diagonal principal da matriz, os ponteiros criam uma lista duplamente encadeada, sendo que os ponteiros da direita e esquerda ordenam crescente e decrescente, respectivamente, os vértices adjacentes a i pelo custo da aresta (i, j) . A Figura 1 ilustra como os dados são mantidos na estrutura de dados proposta. Nesta Figura foi representado apenas um dos ponteiros para facilitar a visualização. A matriz, à direita, apresenta os vértices e, entre parênteses, os custos das arestas.

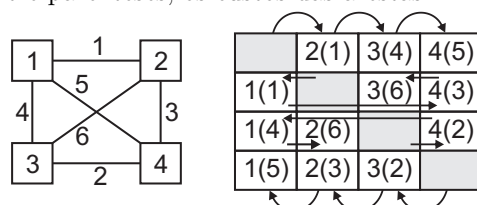


Figura 1. Nova estrutura de dados para armazenar um grafo.

3.2 Representação de uma solução

A representação de uma solução para problemas de roteamento de veículos pode ser feita através de um conjunto de vetores, cada um indicando uma rota e armazenando os consumidores na ordem em que devem ser visitados.

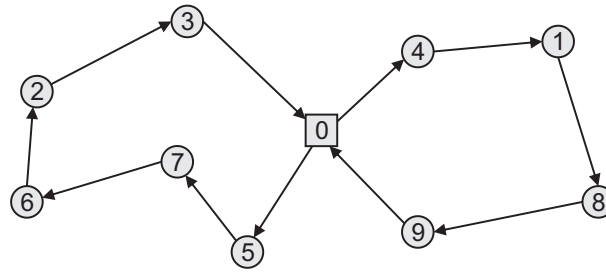
Porém, nos problemas de roteamento multiobjetivo com coleta seletiva, além da ordem em que os consumidores são visitados, deve-se armazenar quais demandas de coleta serão atendidas. Para isso, utiliza-se um vetor binário informando, para cada consumidor, o estado de sua demanda de coleta, atendida ou não atendida. A Figura 2 apresenta uma solução para o problema de roteamento de veículos multiobjetivo com coleta seletiva. Nessa ilustração, os dois primeiros vetores mostram as rotas que compõem a solução, indicando a ordem em que os consumidores devem ser visitados, sempre começando e terminado no depósito, representado pelo número 0. O último vetor informa o estado da demanda de coleta de cada consumidor, sendo representada pelo valor 1, demanda de coleta atendida, ou 0, demanda de coleta não atendida.

3.3 Análise da viabilidade das rotas

Durante a construção, ou na alteração de uma solução de um problema de roteamento de veículos com serviços de coleta e entrega de mercadorias, é necessário verificar se o atendimento de uma demanda não extrapola a capacidade máxima do veículo.

Uma implementação simples para este procedimento pode ser feita calculando-se a carga do veículo após o atendimento de cada demanda dos consumidores da rota modificada, verificando, assim, se a capacidade do veículo não será extrapolada com a inserção das demandas de entrega e coleta do novo consumidor. Apesar de sua fácil implementação, o desempenho do algoritmo para geração de rotas válidas pode ser prejudicado, já que este processo tem complexidade $O(l)$, sendo l o comprimento da rota.

Como a avaliação da viabilidade é realizada inúmeras vezes, qualquer aperfeiçoamento realizado melhora o desempenho dos algoritmos que o utilizam. Assim, é proposto o armazenamento de algumas informações



Rota 1: [0-4-1-8-9-0] - **Rota 2:** [0-5-7-6-2-3-0]
Estado Coleta: [1-1-0-1-0-0-1-1-1]

Figura 2. Representação de uma solução para o problema de roteamento de veículos multiobjetivo com coleta seletiva.

adicionais sobre a carga do veículo ao longo da rota, tornando mais eficiente esta análise de viabilidade. O Algoritmo 1 apresenta o pseudo-código do procedimento proposto para análise de viabilidade das rotas.

Algoritmo 1: Análise de viabilidade.

```

// Verifica a carga do veículo ao partir e retornar ao depósito
1 se ( $SomaEntregas + E(x)$ ) >  $Q$  então
2   retorna falso;
3 se ( $SomaColetas + C(x)$ ) >  $Q$  então
4   retorna falso;
// Verifica se o local onde o consumidor será inserido está antes do ponto de maior
// carga
5 se  $p \leq maxPosicao$  então
6   se ( $infoCarga[maxPosicao] + C(x)$ ) >  $Q$  então
7     retorna falso;
8   se  $E(x) > C(x)$  então
9     para  $i = 0$  até  $p - 1$  fazer
10      se ( $infoCarga[i] + E(x)$ ) >  $Q$  então
11        retorna falso;
12 senão
13   // Consumidor será inserido depois do ponto de maior carga
14   se ( $infoCarga[maxPosicao] + E(x)$ ) >  $Q$  então
15     retorna falso;
16   se  $E(x) < C(x)$  então
17     para  $i = p$  até  $l - 1$  fazer
18       se ( $infoCarga[i] + C(x)$ ) >  $Q$  então
19         retorna falso;
19 retorna verdadeiro;

```

Sejam $E(x)$ e $C(x)$ as demandas de entrega e coleta do consumidor x , respectivamente. O Algoritmo 1 analisa, inicialmente, se a carga do veículo não extrapola a capacidade máxima (Q), quando o veículo parte do depósito e quando ele retorna ao depósito (linhas 2–7). Em seguida, verifica se a posição onde o veículo apresenta maior carga está antes ou depois da posição p que se deseja inserir o consumidor x (linha 9).

Caso o consumidor seja inserido antes da posição na qual o veículo atinge o ponto de maior carga ($maxPosicao$), então a carga máxima do veículo será acrescida apenas da demanda de coleta do consumidor. O vetor *infoCarga* armazena a carga presente no veículo em cada posição da rota (linhas 6–7). A Figura reffig:viabilidade11.a exemplifica uma rota na qual o depósito está representado por um quadrado, os consumidores representados por circunferências, as demandas de entrega e coleta de cada consumidor representadas entre parênteses, respectivamente. O valor nas arestas indica a carga presente no veículo

(*infoCarga*), sendo a carga máxima do veículo destacada em negrito. Na Figura 3.b, pretende-se inserir um vértice (6) antes do ponto no qual o veículo atinge sua carga máxima (9). Na situação ilustrada, a carga máxima do veículo será acrescida apenas da demanda de coleta do consumidor 6, $C(6) = 4$.

Se a carga máxima do veículo acrescido da demanda de coleta do consumidor inserido for maior que a capacidade do veículo, então o consumidor x não pode ser inserido na posição p (linhas 6–7). Caso contrário, o algoritmo verifica se a demanda de entrega $E(x)$ é menor que a demanda de coleta $C(x)$ (linhas 8–11). Desta maneira, garante-se que a carga máxima do veículo permanecerá na mesma aresta e, assim, a inserção no consumidor não inviabiliza a solução. Senão, torna-se necessário verificar a carga do veículo no trajeto entre o depósito e o vértice inserido, pois a carga máxima do veículo pode agora estar neste intervalo.

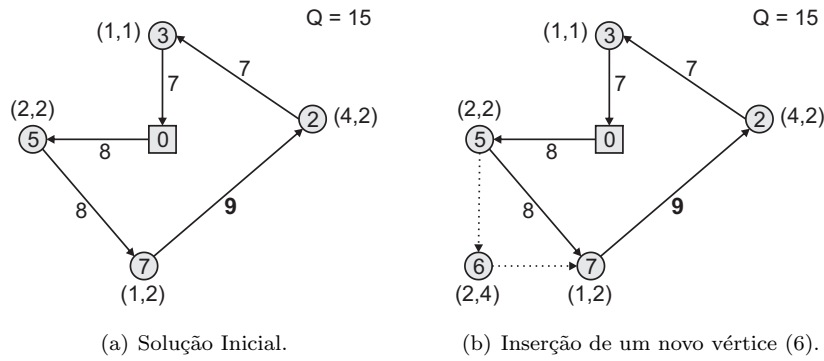


Figura 3. Análise de viabilidade.

Quando o ponto de inserção p estiver após o ponto de maior carga, o procedimento é análogo (linhas 12–18). No fim, se o veículo não extrapola sua capacidade máxima, então é retornado *verdadeiro*, indicando que a inserção do consumidor na posição p é viável (linha 19).

Neste procedimento, quando $p \leq \text{maxPosicao}$ e $E(x) \leq C(x)$ ou $p > \text{maxPosicao}$ e $C(x) \leq E(x)$, tem-se o melhor caso, com complexidade $O(1)$, pois ele não executa nenhum bloco de repetição. Apenas no pior caso o Algoritmo 1 apresenta complexidade $O(l)$.

O pior caso ocorre em duas situações: (i) quando se trata da inserção de um consumidor x imediatamente anterior à posição de maior carga, a posição de maior carga ocorre no final da rota e a demanda de entrega do consumidor x é maior que a demanda de coleta x ; (ii) quando ocorre a inserção de um consumidor x logo após a posição de maior carga e a posição de maior carga ocorre no início da rota, e a demanda de coleta do consumidor x é maior que a demanda de coleta de x . Em ambos casos, as linhas 9–11 ou 16–18 serão executadas aproximadamente l vezes.

3.4 Busca local iterativa multiobjetivo (MOILS)

A meta-heurística busca local iterativa (ILS, *Iterated Local Search*) consiste em aprimorar o procedimento de busca local gerando novas soluções de partidas através de mecanismos de perturbações aplicados à solução ótima local (Lourenço et al., 2003).

Ao se tratar de problemas de roteamento de veículos mono-objetivo com serviços de coleta e entrega, heurísticas baseadas no ILS apresentam bons resultados, sem a necessidade de ajuste de muitos parâmetros (Subramanian et al., 2010). Além disso, adaptações desta meta-heurística, quando utilizadas para resolução de outros problemas de otimização combinatória multiobjetivo, também apresentam bons resultados (Geiger, 2006). Assim, este capítulo irá apresentar uma heurística baseada na meta-heurística ILS para resolução de problemas multiobjetivo utilizando o conceito de Dominância de Pareto, possibilitando, assim, a resolução de problemas de otimização combinatória multiobjetivo, em especial o MOVRPSP. À adaptação proposta é dado o nome de busca local iterativa multiobjetivo (MOILS, *Multiobjective Iterated Local Search*).

O algoritmo proposto possui 5 etapas: (i) geração de uma solução inicial; (ii) seleção de uma solução da fronteira de Pareto aproximada para ser explorada; (iii) busca local, que retorna uma solução melhorada; (iv) perturbação, que modifica uma solução; e (v) atualização da fronteira de Pareto aproximada dada uma solução para o problema.

O funcionamento desta heurística é ilustrado no Algoritmo 2. O MOILS inicia seu funcionamento com a geração de soluções iniciais (linha 1). As soluções geradas são inseridas no conjunto *Front* que mantém um conjunto de soluções não dominadas. Em seguida, são executados *maxIter* iterações, onde uma solução em *Front* é selecionada e o bloco de repetição principal do algoritmo (linhas 6–15) é executado. Neste bloco, as etapas de perturbação (linha 7) e busca local (linha 8), semelhantes ao ILS, são aplicadas à solução escolhida.

Dada a solução gerada, o conjunto *Front* é atualizado (linha 9). Se a solução gerada após a perturbação e busca local for inserida em *Front* (indicada pela variável lógica *inserido*), então *cont* é reiniciado (linha 11); senão *cont* é incrementado de uma unidade (linha 14). O parâmetro *maxCount* representa o número máximo de vezes que uma solução será explorada sem a inserção de uma solução não dominada no conjunto *Front*.

Algoritmo 2: Algoritmo MOILS.

Entrada: *maxIter*
Entrada: *maxCount*

```

1 Front  $\leftarrow$  gerarSoluçõesIniciais();
2 iter  $\leftarrow$  0;
3 enquanto iter < maxIter faça
4   s'  $\leftarrow$  selecionar(Front);
5   cont  $\leftarrow$  0;
6   enquanto cont < maxCount faça
7     s''  $\leftarrow$  perturbação(s');
8     s''  $\leftarrow$  buscaLocal(s'');
9     inserido  $\leftarrow$  atualizar(Front, s'');
10    se inserido então
11      cont  $\leftarrow$  0;
12      s'  $\leftarrow$  s'';
13    senão
14      cont  $\leftarrow$  cont + 1;
15  iter  $\leftarrow$  iter + 1;
16 retorna Front;

```

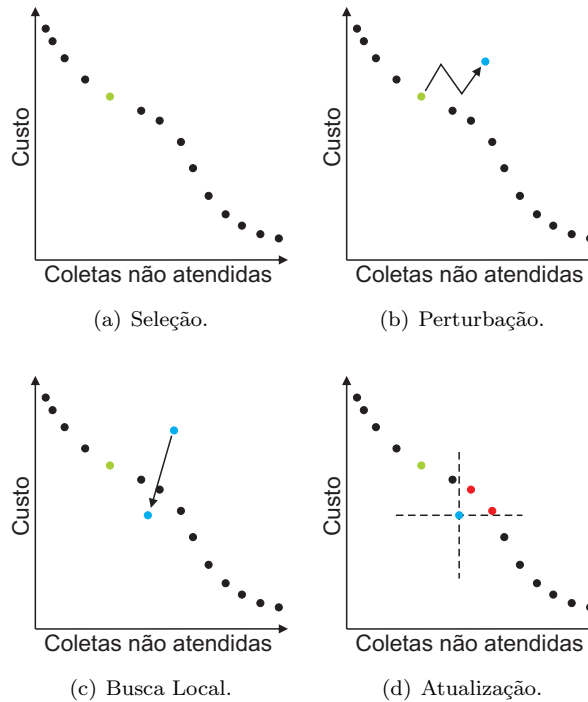


Figura 4. Etapas do algoritmo MOILS.

A Figura 4 ilustra as etapas do algoritmo MOILS. Na Figura 4.a, uma solução contida na fronteira de Pareto aproximada é selecionada. Nas Figuras 4.b e 4.c, esta solução passa pelos procedimentos de perturbação e busca local, respectivamente. Por fim, a fronteira é atualizada com a nova solução gerada, Figura 4.d.

Os procedimentos de geração da solução inicial, seleção, busca local, perturbação e atualização são detalhados nas subseções seguintes.

3.4.1 Solução inicial

A solução inicial para o MOVRPSP consiste em encontrar uma solução para o problema de roteamento de veículos com coleta e entrega simultânea (VRPSPD) e para o problema de roteamento de veículo capacitado (CVRP, *Capacitated Vehicle Routing Problem*), sendo estas soluções extremos da fronteira de Pareto aproximada. A solução para ambos problemas é obtida aplicando o algoritmo baseado no ILS, proposto por Penna et al. (2012). Assim sendo, o ILS é executado duas vezes, sendo uma para solucionar o VRPSPD, no qual todas as demandas de coleta do problema são consideradas e outra para solucionar o CVRP, desconsiderando as demandas de coleta dos consumidores.

As soluções encontradas representam os pontos extremos da fronteira de Pareto aproximada, conforme Figura 5. Uma vez encontrados os extremos e sendo estes de boa qualidade, os demais passos do algoritmo consistem em encontrar as soluções contidas no intervalo entre os dois pontos extremos já definidos.

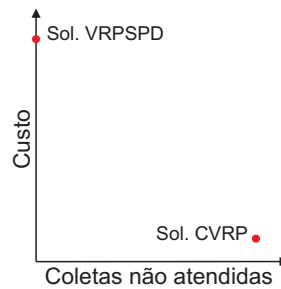


Figura 5. Pontos extremos da fronteira de Pareto aproximada.

3.4.2 Seleção

O procedimento de seleção utilizado é baseado na *crowding-distance* do algoritmo NSGA-II, proposto por Deb et al. (2002). Quanto mais distante das outras soluções uma determinada solução estiver, maior a probabilidade desta ser escolhida. Assim, é possível realizar a exploração do espaço de soluções de forma mais inteligente, onde é dada prioridade a regiões pouco exploradas, evitando a concentração da busca em determinados nichos.

Neste trabalho, o cálculo da *crowding-distance* é um pouco diferente do realizado no NSGA-II. No MOILS as soluções presentes nos extremos da fronteira têm a *crowding-distance* igual a duas vezes a distância entre elas e a solução mais próxima, enquanto no NSGA-II, as soluções presentes nas extremidades da fronteira recebem um valor infinito. Esta alteração é necessária já que após a geração das soluções iniciais, a fronteira é composta apenas pelas duas soluções que se encontram nas extremidades da fronteira (solução do CVRP e solução do VRPSPD). Assim, para reduzir a possibilidade de serem demasiadamente selecionadas durante todo o procedimento, o valor da *crowding-distance* dessas soluções precisa ser reduzido.

3.4.3 Busca Local

A etapa de busca local realiza a exploração da vizinhança de uma solução com o objetivo de encontrar outras soluções não dominadas para o problema. No algoritmo apresentado, foi utilizado o método busca em descida em vizinhança variável aleatória (RVND, *Randomized Variable Neighborhood Descent*), proposta por Penna et al. (2012), que explora uma solução aplicando aleatoriamente diferentes estruturas de vizinhanças.

Neste trabalho, foram utilizadas um conjunto de estruturas de vizinhanças que podem ser divididas em dois grupo: inter-rotas e intra-rota. As estruturas de vizinhança inter-rotas são aquelas que envolvem duas ou mais rotas no movimento. Já as intra-rotas envolvem apenas uma única rota. A busca realizada em todas as estruturas de vizinhança foi baseada na estratégia melhor aprimorante que avalia todas as soluções vizinhas antes do movimento ser aplicado. As Subseções 3.4.3.1 e 3.4.3.2 descrevem estas estruturas.

3.4.3.1 Estruturas de vizinhança inter-rotas

Foram implementadas dois tipos de estruturas de vizinhança inter-rotas: que exploram soluções levando em consideração apenas o objetivo custo de transporte e outras que consideram, além do custo de transporte, o objetivo de minimização das demandas de coleta não atendidas.

As seguintes buscas locais visam a redução do custo de transporte e podem ser aplicadas a diversos problemas de roteamento mono-objetivo.

- **Shift(1,0):** um consumidor é transferido de uma rota para outra rota. Na Figura 6.b, o consumidor 5 é removido de sua rota e inserido em outra rota.

- **Shift(2,0):** dois consumidores adjacentes são transferidos de uma rota para outra rota, continuando adjacentes, mas não necessariamente na mesma ordem. Na Figura 6.c, os consumidores 5 e 6 são removidos e inseridos em outra rota.
- **Swap(1,1):** permutação entre dois consumidores de duas rotas distintas. Na Figura 6.d, os consumidores 3 e 5 são permutados.
- **Swap(2,1):** permutação de dois consumidores adjacentes de uma rota com um consumidor de outra rota. Os dois consumidores adjacentes removidos devem permanecer adjacentes quando inseridos na outra rota, porém não necessariamente na mesma ordem. Na Figura 6.e, o consumidor 3 é trocado com os consumidores 4 e 5.
- **Swap(2,2):** permutação de dois consumidores adjacentes de uma rota com outros dois consumidores, também adjacentes, de outra rota. Assim como na Swap(2,1), os consumidores adjacentes devem permanecer adjacentes mas não necessariamente na mesma ordem. Na Figura 6.f, os consumidores 4 e 5 são trocados com os consumidores 2 e 3.
- **Crossover:** uma aresta (i, j) de uma rota r_1 e outra aresta (i', j') de uma rota r_2 são removidas. Em seguida as arestas (i, j') e (i', j) são inseridas. Na Figura 6.g, as arestas $(3, 0)$ e $(5, 6)$ são removidas e as arestas $(3, 6)$ e $(5, 0)$ são inseridas.
- **TryEmptyRoute:** remove os consumidores de uma determinada rota e tenta reinseri-los nas demais rotas pertencentes à solução a fim e reduzir o número de veículos utilizados na solução.

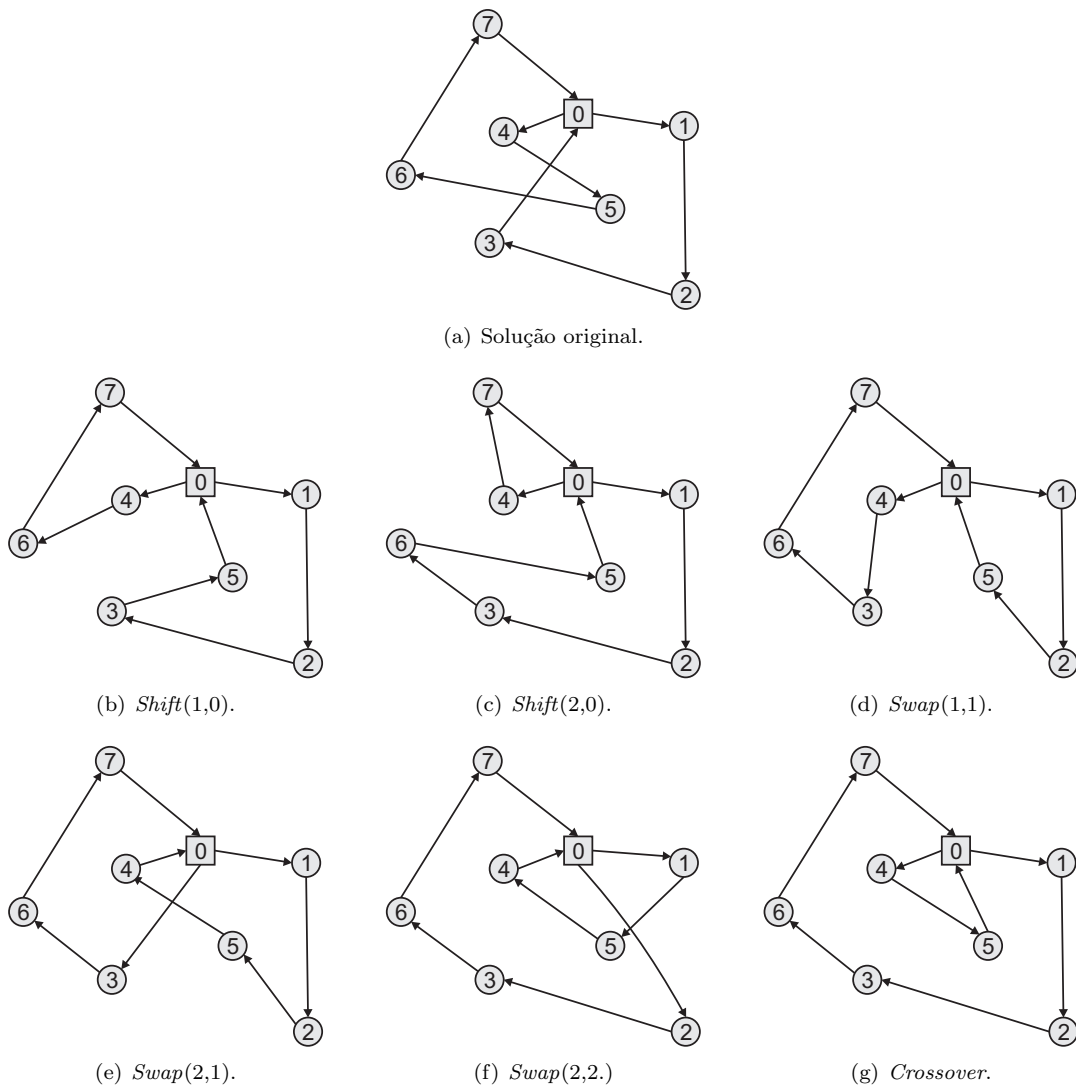


Figura 6. Estruturas de vizinhança inter-rotas.

Devido à natureza multiobjetivo do MOVRPSP, proposto neste trabalho, faz-se necessária a utilização de estruturas de vizinhança que atuem nos outros objetivos e não apenas no custo de transporte. Este capítulo apresenta duas estruturas inter-rotas que atuam tanto na redução do custo de transporte quanto na minimização das demandas de coletas não atendidas. O objetivo é melhorar a diversidade das soluções e aproximá-las da fronteira de Pareto ótima.

Estas estruturas de vizinhanças são adaptações da $Shift(1,0)$ e $Swap(1,1)$, descritas anteriormente. O procedimento é similar, modificando apenas o estado da demanda de coleta (satisfeita ou não satisfeita) dos consumidores envolvidos no movimento. Assim, ao invés de retornar uma única solução, retornarão um conjunto de soluções não dominadas.

- **MOShift(1,0):** um consumidor é transferido de uma rota para outra rota. A transferência é feita considerando o estado de coleta atual do consumidor e, em seguida, a busca local é aplicada novamente alterando o estado de coleta do consumidor envolvido no movimento. Logo, para cada consumidor pertencente à solução, duas soluções vizinhas são encontradas.
- **MOSwap(1,1):** permutação entre dois consumidores de duas rotas distintas. A permutação é feita considerando os estados atuais das demandas de coleta dos consumidores e alternando estes estados. Logo, para cada consumidor pertencente à solução, quatro novas soluções vizinhas são geradas: uma considerando os estados atuais das demandas de coleta de ambos consumidores, a segunda alterando o estado da demanda de coleta de um dos consumidores, a terceira alterando o estado do outro consumidor e, por fim, alterando os estados das demandas de coletas dos dois consumidores envolvidos no movimento.

A Figura 7 exemplifica os movimentos $MOShift(1,0)$ e $MOSwap(1,1)$. Nesta figura, o valor nas arestas representa a carga do veículo naquele ponto e os valores entre parênteses indicam a demanda de entrega e coleta de cada consumidor, respectivamente. As Figuras 7.b e 7.c mostram as duas possíveis soluções retornadas pela estrutura de vizinhança $MOShift(1,0)$. Na Figura 7.b, o consumidor 3 é realocado em outra rota desconsiderando a sua demanda de coleta e, na Figura 7.c, o consumidor 3 é realocado considerando o atendimento desta demanda. As Figuras 7.d, 7.e, 7.f e 7.g representam as quatro soluções geradas pela estrutura de vizinhança $MOSwap(1,1)$. Na Figura 7.d, os consumidores 3 e 5 são permutados desconsiderando a demanda de coleta de ambos. Na Figura 7.e, estes mesmos consumidores são permutados desconsiderando a demanda de coleta do consumidor 3 e, na Figura 7.f, desconsiderando a demanda de coleta do consumidor 5. Na Figura 7.g, estes consumidores são permutados considerando o atendimento das demandas de coleta de ambos. Nas duas estruturas de vizinhança, dadas as soluções geradas, apenas as soluções não dominadas são retornadas.

Para evitar que movimentos inter-rota sejam executados desnecessariamente, uma estrutura de dados auxiliar, proposta por Penna et al. (2012), foi utilizada indicando se uma determinada busca local poderá ser aplicada à uma rota específica. Se, em uma iteração anterior, a estrutura de vizinhança i foi aplicada à rota j e não houve nenhuma melhoria da solução, então a estrutura de vizinhança i fica desabilitada para a rota j até que esta rota sofra alguma modificação.

3.4.3.2 Estruturas de vizinhança intra-rota.

Um dos objetivos das estruturas de vizinhança intra-rota é reduzir os custos de transporte das rotas. Assim, quando as estruturas de vizinhança inter-rotas não puderem gerar soluções na fronteira de Pareto aproximada, aplica-se, então, as estruturas de vizinhança intra-rota a fim de reduzir ainda mais os custos de transporte. As estruturas de vizinhança intra-rota implementadas são descritas a seguir.

- **Or-Opt:** um consumidor é inserido em outra posição da mesma rota. Na Figura 8.b, o consumidor 5 é alterado de posição.
- **Or-Opt 2:** dois consumidores adjacentes são removidos e inseridos em outra posição da mesma rota. Na Figura 8.c, os consumidores 4 e 5 são removidos e inseridos em outra posição.
- **Or-Opt 3:** três consumidores adjacentes são removidos e inseridos em outra posição da mesma rota. Na Figura 8.d, os consumidores adjacentes 2, 3 e 4 são removidos e inseridos em outra posição.
- **2-Opt:** duas arestas (i, j) e (i', j') não consecutivas da mesma rota são removidas. Em seguida as arestas (i, i') e (j, j') são inseridas. Na Figura 8.e, as arestas $(3, 4)$ e $(5, 6)$ são removidas e as arestas $(3, 5)$ e $(4, 6)$ são inseridas.
- **Exchange:** realiza a permutação entre dois consumidores de uma mesma rota. Na Figura 8.f, os consumidores 3 e 6 são permutados.

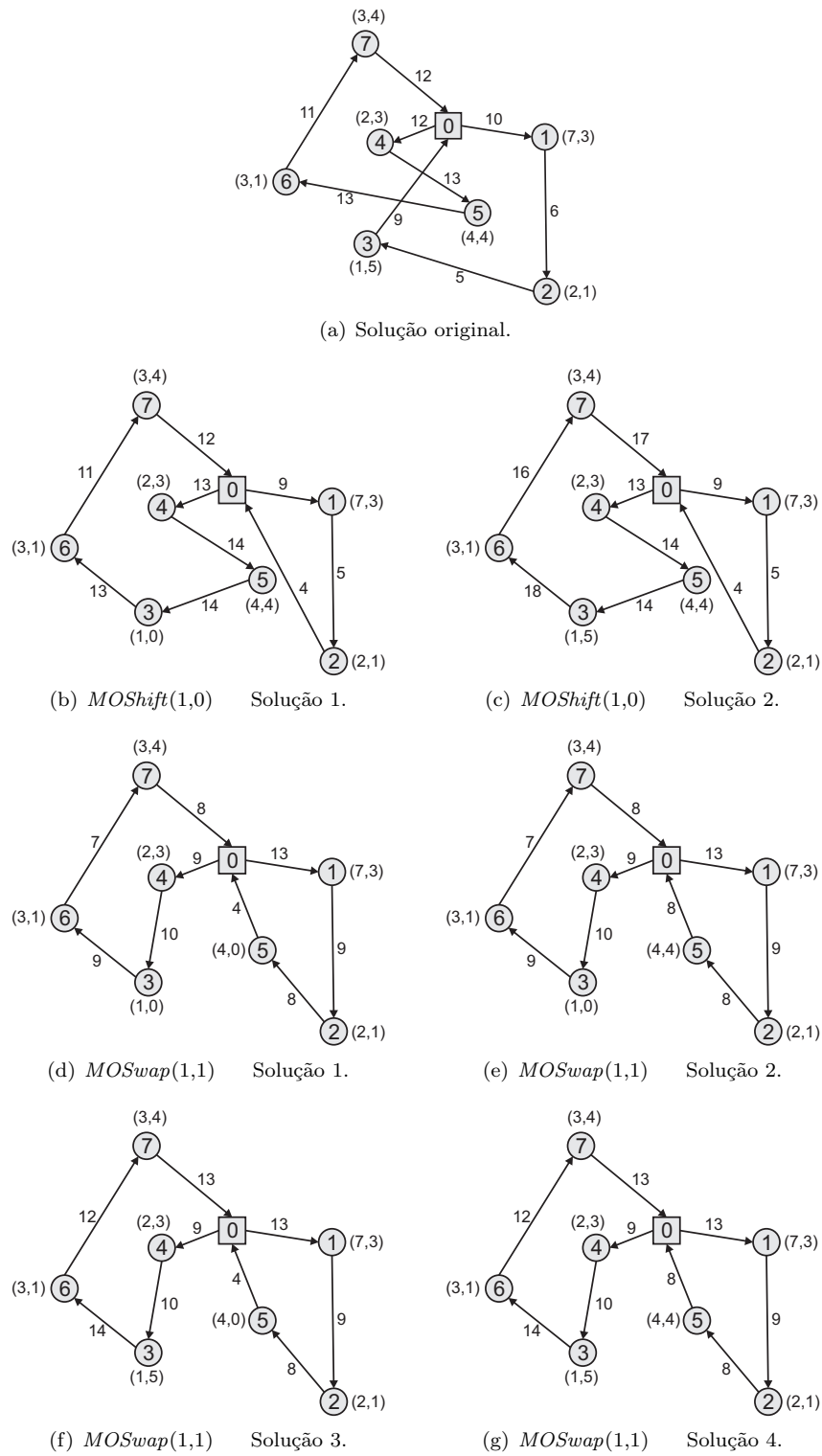


Figura 7. Estruturas de vizinhança inter-rotas.

- **Reverse:** inverte a direção da rota visando reduzir a carga máxima no veículo. Na Figura 8.g, todas as arestas têm a direção alterada, invertendo o sentido da rota.

Para reduzir o número de demandas de coleta não atendidas, foi criada uma estrutura de vizinhança denominada *Adiciona Coleta*. Esta estrutura verifica se a demanda de coleta de cada consumidor pode ser satisfeita sem extrapolar a capacidade do veículo, dada a demanda de coleta deste consumidor. Desta forma, o consumidor não precisará ser realocado em outra posição ou rota.

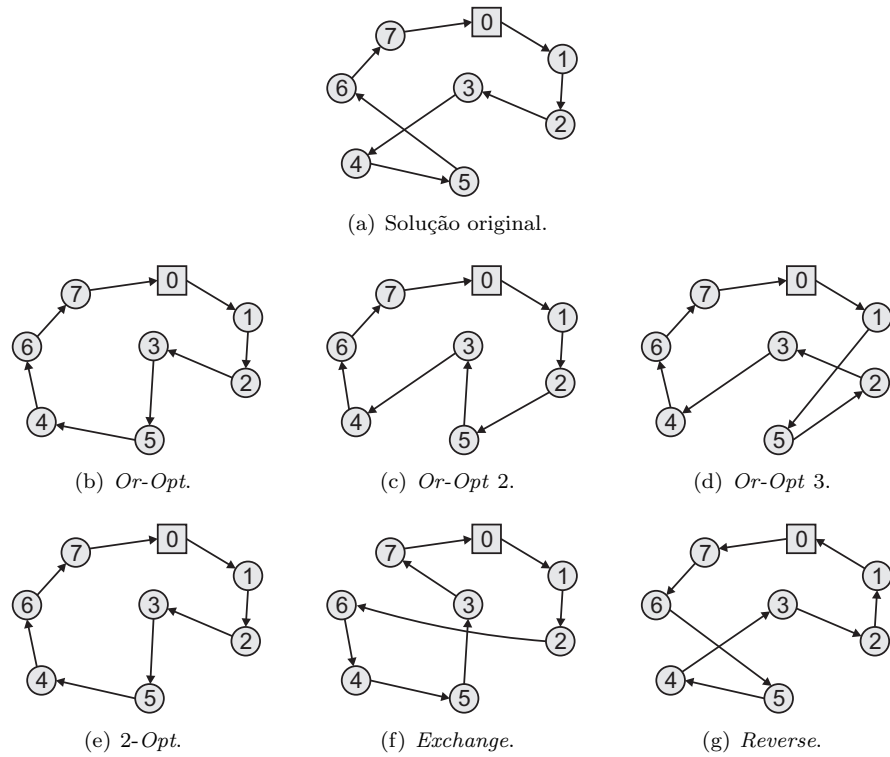


Figura 8. Estruturas de vizinhança intra-rotas.

3.4.4 Perturbação

Na etapa de perturbação do algoritmo são utilizados mecanismos que realizam modificações em uma solução, escapando de ótimos locais, permitindo a exploração de outras áreas do espaço de busca. Neste trabalho, foram criadas perturbações que modificam as soluções em relação a ambos os objetivos do problema de roteamento de veículos multiobjetivo com coleta seletiva.

Assim sendo, o método de perturbação do algoritmo MOILS consiste em aplicar, primeiramente, o procedimento *Altera Coleta* que visa a modificação da solução em relação ao atendimento da demanda de coleta. Neste método, se o consumidor tiver a sua coleta realizada, então esta demanda não será mais atendida, e vice-versa. Quando a demanda de coleta de um consumidor deixar de ser atendida, o consumidor permanece na mesma posição. Caso contrário, ao passar a atender uma demanda de coleta, a capacidade do veículo pode ser extrapolada, inviabilizando a solução. Nestes casos, procura-se uma posição viável para inserir o consumidor. Caso não seja encontrada nenhuma posição viável, uma nova rota é criada para que este consumidor possa ser inserido.

Após a realização do procedimento *Altera Coleta*, a etapa de perturbação do algoritmo MOILS seleciona aleatoriamente e executa, ainda, um dos mecanismos de perturbação, descritos a seguir, que modificam a posição dos consumidores na solução.

- **Multiple Swap:** realiza trocas entre consumidores de rotas distintas escolhidos aleatoriamente;
- **Multiple Shift:** realoca um consumidor escolhido aleatoriamente de uma rota para outra rota;
- **Ejection Chain:** realoca um consumidor de uma rota R_1 em uma rota R_2 , outro da rota R_2 em uma rota R_3 , e assim sucessivamente até que um consumidor da última rota seja inserido na primeira rota.
- **Or-Opt:** mover 1 a 3 consumidores consecutivos de uma rota para outra.

Em todos estes quatro mecanismos de perturbação listados, a inserção dos consumidores é feita na primeira posição viável encontrada.

3.4.5 Atualização

A etapa de atualização verifica se uma determinada solução s não é dominada pelas soluções presentes na fronteira de Pareto aproximada, no Algoritmo 2 representada pelo conjunto *Front*. Se s não é dominada por nenhuma solução em *Front*, então ela é inserida no conjunto e as soluções por ela dominadas são removidas.

3.5 Heurística P_ϵ -ILS

Um método bastante comum para solucionar problemas com múltiplos objetivos baseia-se na transformação deste problema em diversos problemas com um único objetivo, onde as soluções ótimas destes problemas representam as soluções da fronteira de Pareto. Estes métodos definem um conjunto de problemas parametrizados de diferentes formas, sendo que os valores nos quais se parametrizam estes problemas determinarão quais soluções do conjunto Pareto serão alcançadas. Desta forma, a tarefa de escolher esta sequência de valores de parâmetros é extremamente difícil. Em muitos casos é impossível definir valores tais que toda a fronteira de Pareto seja encontrada.

O método ϵ -Restrito, proposto por [Chankong & Haimes \(1983\)](#), é baseado na escalarização no qual um dos objetivos é minimizado, enquanto os demais objetivos são limitados por meio de restrições. As soluções pertencentes a fronteira de Pareto são obtidas através de uma variação sistemática do limite destas restrições.

Definição 4 *Seja m o número de objetivos para determinado problema multiobjetivo, se $x^* \in \mathcal{F}_x$ é eficiente, então existem $i \in \{1, 2, \dots, m\}$ e números reais ϵ_j , $j = 1, \dots, m$ ($j \neq i$) tais que x^* resolve ([Chankong & Haimes, 1983](#)):*

$$x^* = \arg \min_{x \in \mathcal{F}_x} f_i(x)$$

$$\text{Sujeito a: } \{ f_j(x) \leq \epsilon_j, j = 1, \dots, m (j \neq i) \}$$

Segundo a definição 4, o conjunto de soluções eficientes é constituído por cada solução x^* obtida para cada ϵ_j . Assim, a principal vantagem do método ϵ -Restrito é que o mesmo possibilita encontrar a fronteira de Pareto independente da estrutura do problema multiobjetivo, podendo ele ser não-convexo e discreto ([Chankong & Haimes, 1983](#)).

Baseada nos princípios básicos do método ϵ -Restrito, a heurística P_ϵ consiste em definir, a cada iteração, um conjunto de consumidores no qual suas demandas de coleta serão atendidas. Desta forma, este sub-problema gerado é solucionado utilizando um algoritmo baseado no ILS.

Algoritmo 3: Heurística P_ϵ -ILS.

Entrada: $maxCount$

- 1 $s_{CVRP} \leftarrow$ gerar solução para o CVRP utilizando ILS;
- 2 $s_{VRPSPD} \leftarrow$ gerar solução para o VRPSPD utilizando ILS;
- 3 atualizar($Front$, s_{CVRP});
- 4 atualizar($Front$, s_{VRPSPD});
- 5 $cont \leftarrow 0$;
- 6 **enquanto** $cont < maxCount$ **faça**
- 7 $s'_{CVRP} \leftarrow$ Adiciona Coleta(s_{CVRP});
- 8 Atualizar($Front$, s'_{CVRP});
- 9 **enquanto** $\exists consumidor_j \in s'_{CVRP} \mid \ell_j = 0$ **faça**
- 10 Definir j aleatoriamente;
- 11 $\ell_j \leftarrow 1$;
- 12 $s'_{CVRP} \leftarrow$ Insere uma coleta($consumidor_j, s'_{CVRP}$);
- 13 $s'_{CVRP} \leftarrow$ ILS(s'_{CVRP});
- 14 atualizar($Front$, s'_{CVRP});
- 15 **enquanto** $\exists consumidor_j \in s'_{VRPSPD} \mid \ell_j = 1$ **faça**
- 16 Definir j aleatoriamente;
- 17 $\ell_j \leftarrow 0$;
- 18 $s'_{VRPSPD} \leftarrow$ ILS(s'_{VRPSPD});
- 19 atualizar($Front$, s'_{VRPSPD});
- 20 $cont \leftarrow cont + 1$;
- 21 **retorna** $Front$;

O passo inicial define os pontos de partida do algoritmo (linhas 1 e 2). Estes pontos consistem nas soluções do CVRP e VRPSPD, nos quais nenhuma demanda de coleta é atendida e todas as demandas de coletas são atendidas, respectivamente. A busca local iterativa, apresentada em [Penna et al. \(2012\)](#), foi utilizada para resolver ambos problemas. As duas soluções são, então, incluídas na fronteira de Pareto aproximada.

O algoritmo irá executar $maxCount$ iterações (linhas 4–14), no qual a solução do CVRP será modificada gradativamente até que todas as demandas de coleta sejam satisfeitas. Por outro lado, a solução do VRPSPD será também modificada gradativamente até que nenhuma demanda de coleta seja atendida.

Iniciando pela solução s'_{CVRP} , na linha 7, para cada rota aplica-se a busca local intra-rota *Adiciona Coleta*, definida na Subseção 3.4.3.2. Dessa forma, sem alterar a posição dos consumidores na rota, busca-se atender o máximo de demandas de coleta possível. Dada esta solução, a fronteira *Front* é atualizada aplicando o mesmo procedimento do algoritmo MOILS.

Em seguida, enquanto houver demandas de coleta não atendidas na solução s'_{CVRP} , as demandas de coleta não atendidas serão selecionadas aleatoriamente, uma a uma, para serem satisfeitas. Este procedimento resulta na realocação do consumidor, uma vez que o atendimento de uma demanda de coleta pode acarretar na extrapolção da capacidade do veículo. Assim, este consumidor é realocado na melhor posição encontrada dentre todas as rotas, considerando o custo de transporte. Quando nenhuma posição viável é encontrada, uma nova rota é criada para a inclusão deste consumidor. Para refinar a solução obtida, aplica-se o método ILS (Penna et al., 2012) à esta solução. Finalmente, atualiza-se o conjunto de soluções não dominadas *Front*. A Figura 9.a ilustra os passos da heurística P_ϵ -ILS partindo da solução do CVRP.

O procedimento descrito para a solução s'_{CVRP} é repetido para a solução s'_{VRPSPD} com algumas modificações. Cada demanda de coleta satisfeita é selecionada aleatoriamente, uma a uma, para deixarem de ser atendidas. Este procedimento não requer a realocação do consumidor, uma vez que o mesmo não acarreta em nenhuma inviabilidade da solução. Da mesma forma, para refinar a solução, o método ILS é utilizado e o conjunto *Front* é atualizado. A Figura 9.b mostra as soluções geradas pela heurística P_ϵ -ILS, partindo de uma solução do VRPSPD.

Como a seleção das coletas a serem atendidas ou não atendidas é feita de forma aleatória, o método é executado várias vezes (*maxCount*) a fim de encontrar soluções mais diversificadas e aproximadas da fronteira de Pareto ótima.

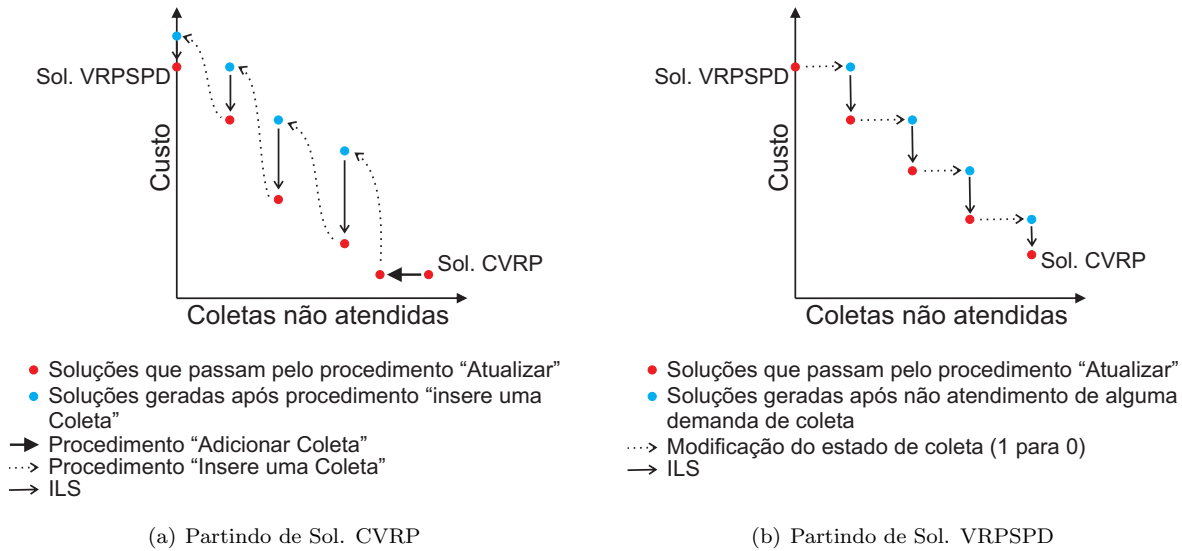


Figura 9. Etapas da heurística P_ϵ -ILS.

3.6 Algoritmo genético de ordenação não-dominante II (NSGA-II)

O algoritmo genético de ordenação não-dominante II (NSGA-II, *Non-Dominated Sorting Genetic Algorithm-II*) foi adaptado a fim de solucionar o MOVRPSP. Os passos do algoritmo seguem os princípios básicos do NSGA-II, proposto por Deb et al. (2002).

Para o MOVRPSP, os indivíduos da população representam uma solução completa do problema tratado, ou seja, um conjunto de rotas e um vetor binário que armazena o estado da demanda de coleta dos consumidores (atendida ou não atendida). O método de geração da população inicial P , assim como nos algoritmos MOILS e P_ϵ -ILS, inicia buscando soluções para o VRPSPD e CVRP para então inseri-las em P . Para uma população com p indivíduos, as demais soluções são obtidas a partir da alteração do estado das demandas de coletas das soluções obtidas para o CVRP e VRPSPD, sendo que metade dos indivíduos da população serão gerados a partir da solução do CVRP e a outra metade a partir do VRPSPD. Iniciando o procedimento pela solução do CVRP, o algoritmo calcula uma taxa ($2n/p$, sendo n é o número de consumidores) no qual as demandas de coleta serão atendidas. Ao término do procedimento, o último indivíduo terá todas ou quase todas as demandas de coleta satisfeitas. Procedimento similar é executado para gerar as soluções iniciais partindo do VRPSPD, sendo que, ao término do procedimento, o último indivíduo definido terá poucas ou nenhuma demanda de coleta satisfeita.

A população filha Q é gerada a partir da recombinação de indivíduos da população P . Dois operadores de cruzamento são considerados neste trabalho, tendo eles igual probabilidade de serem selecionados. Cada indivíduo *filho* pode sofrer mutação, dada uma probabilidade de mutação p_{mut} . Os operadores de mutação utilizados neste trabalho são *Or-opt* e *Altera Coleta*, descritos na Seção 3.4.4. Estes também têm igual probabilidade de serem selecionados para serem aplicados.

A fase de seleção do NSGA-II ocorre em dois momentos distintos do algoritmo. O primeiro deles, após a população inicial ser gerada, o algoritmo utiliza a técnica torneio binário da seguinte forma: dados dois indivíduos a e b , se um indivíduo a domina o indivíduo b , então o indivíduo a é selecionado para o processo de recombinação, caso contrário, o indivíduo b é selecionado. Se a não domina b e b não domina a , ambos indivíduos são selecionados. A partir de então, o método de seleção realiza uma ordenação por não-dominância dos indivíduos pertencentes a $P \cup Q$, retornando um conjunto de fronteiras não dominadas. Seguindo esta ordenação, as fronteiras são incluídas na nova população P , preservando as melhores soluções da população anterior (elitismo). Quando um fronteira não puder ser totalmente incluída em P , os indivíduos dessa fronteira são selecionados de acordo com o método *Crowding-Distance*. Em seguida, a técnica torneio de multidão é aplicada para seleção dos indivíduos a serem recombinados e inseridos na população Q .

No torneio de multidão uma solução s_i , pertencente a fronteira i após a ordenação de dominância, é considerada melhor que a solução s_j , pertencente a fronteira j , se (Deb et al., 2002):

1. s_i possui um nível de não dominância melhor que s_j , ou seja, se $i < j$.
2. $i = j$ e a distância de multidão de s_i é maior que distância de multidão de s_j .

O Algoritmo 4 e a Figura 10 ilustram o funcionamento do NSGA-II implementado.

Algoritmo 4: Pseudocódigo do NSGA-II.

```

1  $g \leftarrow 0$ ; // contador de gerações
2  $P_g \leftarrow$  Gerar População Inicial(); // População Pai
3  $Q_g \leftarrow$  Realizar Seleção Torneio, Cruzamento e Mutação( $P_g$ ); // População Filha
4 repita
5    $R_g \leftarrow P_g \cup Q_g$ ;
6    $F[\ ] \leftarrow$  Ordenação por não-dominância( $R_g$ );
7    $P_{g+1} \leftarrow \emptyset$ ;
8    $i \leftarrow 1$ ;
9   enquanto  $|P_{g+1} + F_i| \leq N$  faça
10     $P_{g+1} \leftarrow P_{g+1} \cup F_i$ ;
11     $i \leftarrow i + 1$ ;
12   para todo solução  $s_j \in F_i$  fazer
13     $d_j \leftarrow$  crowding-distance( $s_j$ );
14    $s \leftarrow$  Ordenar  $F_i$  de forma ascendente de acordo com  $d_j$ ;
15   para  $j = 1$  até  $j = N - |P_{g+1}|$  fazer
16     $P_{g+1} \leftarrow s_j$ ;
17    $Q_{g+1} \leftarrow$  Realizar Seleção Torneio de Multidão, Cruzamento e Mutação( $P_{g+1}$ );
18    $g \leftarrow g + 1$ ;
19 até Critério de Parada;
20  $R_g \leftarrow P_g \cup Q_g$ ;
21  $Fronteira \leftarrow$  Obter soluções não dominadas( $R_g$ );
22 retorna  $Fronteira$ ;
```

3.6.1 Operadores de cruzamento

Neste trabalho foram adotados dois métodos de cruzamento: cruzamento baseado em rota e *split*, descritos a seguir.

3.6.1.1 Cruzamento baseado em rota

O cruzamento baseado em rota (RBX, *Route Based Crossover*) seleciona aleatoriamente algumas rotas de um indivíduo pai_1 para serem incluídas no indivíduo $filho_1$. Em seguida, os demais consumidores não contidos na solução do $filho_1$ são inseridos de acordo com a solução do indivíduo pai_2 . De forma similar, algumas rotas do indivíduo pai_2 são selecionadas aleatoriamente para serem incluídas no indivíduo $filho_2$ e este indivíduo é completado de acordo com as rotas do indivíduo pai_1 .

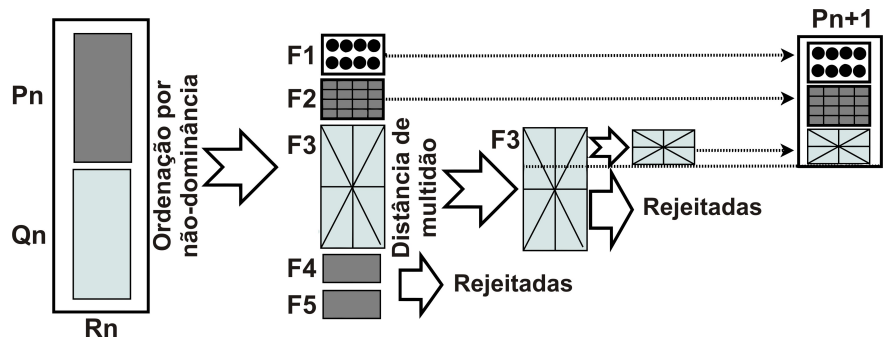


Figura 10. Algoritmo NSGA-II.

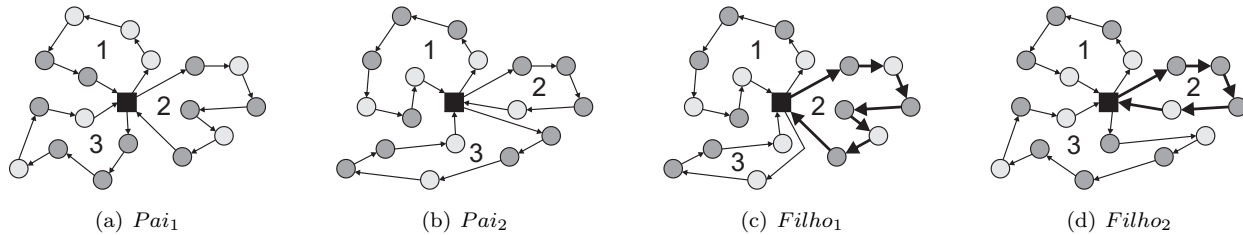


Figura 11. Cruzamento baseado em rota (RBX).

Este método é utilizado por recombinar tanto a posição dos consumidores nas rotas quanto o estado de coleta dos consumidores. A Figura 11 exemplifica o funcionamento do método de cruzamento RBX. O quadrado negro indica a posição do depósito e as circunferências na cor clara representam os consumidores, tais que suas demandas de coletas são atendidas, e as circunferências na cor escura representam os consumidores com coletas não satisfeitas. As Figuras 11.a e 11.b são os dois indivíduos (Pai_1 e Pai_2) selecionados para o cruzamento. O indivíduo $Filho_1$ é gerado a partir da recombinação dos indivíduos Pai_1 e Pai_2 , no qual a rota 2 é totalmente mantida do Pai_1 e as demais são inseridas do Pai_2 . Da mesma forma, o indivíduo $Filho_2$ mantém a rota 2 do Pai_2 e o restante é inserido conforme Pai_1 . Quando um consumidor é copiado de uma solução Pai para uma solução $Filho$, o estado de coleta desse consumidor é mantido do indivíduo Pai .

3.6.1.2 Cruzamento *split*

O método de cruzamento *split*, proposto por Prins (2004), consiste da transformação dos indivíduos *pais* em soluções para o problema do caixeiro viajante (TSP, *Traveling Salesman Problem*). A estas soluções, aplica-se o método de cruzamento sequencial (OX) e, então, a solução para o TSP é novamente transformada em uma solução para o VRP através de um procedimento denominado *split*.

O processo de transformação de um problema do caixeiro viajante em um problema de roteamento de veículo, inicialmente, constrói um grafo direcionado acíclico (DAG), $D=(V,A)$ com $V=\{0,1,...,n\}$, sendo n o número de consumidores e o vértice 0 representando o depósito. As arestas contidas na DAG representam diferentes rotas viáveis, definidas como: para todo vértice i e j pertencente V , tal que $i < j$, o arco (i,j) pertence ao conjunto de arestas A se a capacidade do veículo nesta rota não é extrapolada.

A Figura 12 mostra como gerar um grafo direcionado acíclico (Figura 12.b) a partir de uma solução do problema do caixeiro viajante (Figura 12.a). Na solução do TSP (Figura 12.a), os números indicados entre parênteses representam a demanda de entrega de cada consumidor e os valores nas arestas representam o custo de percorrê-la. Na DAG, cada aresta indica uma rota possível de ser gerada a partir da solução do TSP. Por exemplo, na aresta $a : 40$ indica que a rota $0 - a - 0$ tem custo 40, a aresta $bcd : 120$ indica que a rota $0 - b - c - d - 0$ tem custo 120. A aresta $abc : 90$, por exemplo, não aparece na DAG, pois a carga do veículo extrapola sua capacidade máxima ($Q=10$).

Assim sendo, dado o grafo direcionado acíclico, busca-se, então, o caminho mais curto neste grafo. Na Figura 12.b, os valores indicados acima do vértice mostram o valor do caminho mais curto de alcançar este vértice a partir do vértice 0. O valor 205 acima do último vértice indica que o caminho mais curto para alcançá-lo, partindo do vértice 0, tem custo 205, sendo que este caminho passa pelas arestas: $ab : 55, c : 60, de : 90$.

Portanto, dado o caminho mais curto, é possível definir as partições que serão aplicadas à solução do problema do caixeiro viajante. Serão criadas as seguintes rotas: $0 - a - b - 0$, $0 - c - 0$ e $0 - d - e - 0$, como mostrado na Figura 12.c.

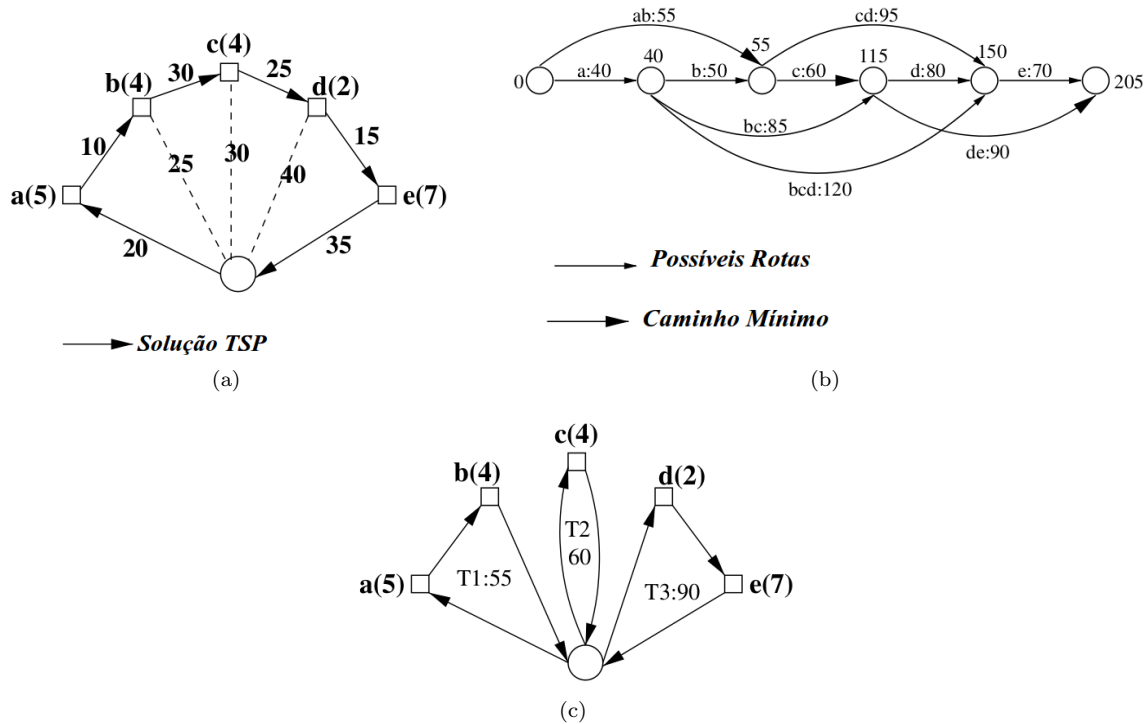


Figura 12. Transformando uma solução do TSP em uma solução para o VRP, de acordo com o cruzamento *split* (Prins, 2004).

4. Resultados Experimentais

Para avaliar o desempenho dos algoritmos MOILS, NSGA-II e P_ϵ -ILS, os testes foram realizados a partir da execução de cada algoritmo em um conjunto de 14 instâncias contendo entre 50 e 199 consumidores, mais o depósito (Salhi & Nagy, 1999). O custo de transporte é calculado como a distância Euclidiana entre os pontos. Nos problemas CMT1 a CMT5, os consumidores estão aleatoriamente distribuídas no plano, enquanto nos problemas CMT11 e CMT12, estes aparecem em grupos, no qual o depósito não está centralizado.

Os parâmetros livres dos algoritmos comparados foram ajustados com o objetivo de proporcionar a cada método, aproximadamente, o mesmo tempo de execução. Uma análise estatística similar à empregada nesta seção não apontou diferenças significativas no tempo de execução dos algoritmos.

Para solucionar os problemas teste, foram atribuídos os seguintes valores aos parâmetros livres dos algoritmos:

1. MOILS: após testes preliminares, às constantes do algoritmo foram atribuídos os seguintes valores: $maxIter = 100$ e $maxCount = 15$;
2. P_ϵ : testes preliminares sugeriram $maxCount = 50$;
3. NSGA-II: O número de gerações do algoritmo e o tamanho da população foi definido como 250 e 500, respectivamente. A taxa de mutação (p_{mut}) é igual a 0,3;
4. ILS: o algoritmo, baseado no ILS, utilizado em todos os algoritmos para encontrar soluções dos problemas de roteamento de veículos mono-objetivo foi definido conforme apresentado por Penna et al. (2012).

Para avaliar o desempenho relativo para os três algoritmos, três métricas foram consideradas: Cardinalidade (Card), Hipervolume (S), Cobertura de dois conjuntos (CS) que são definidos a seguir.

- Cardinalidade (Card): número de soluções contidas na fronteira de Pareto.
- Hipervolume (S): é dado pela soma dos hipercubos formados pelas soluções não dominadas obtidas pelos algoritmos. Esta métrica calcula o volume da região coberta entre os pontos das soluções na fronteira encontrada e um determinado ponto de referência (W). Para problemas de minimização, maiores hipervolumes indicam melhores soluções.

- Cobertura (C): dadas duas fronteiras de Pareto aproximadas A e B, $C(A,B)$ calcula o número de soluções em B que são fracamente dominadas por outra fronteira A, dada a cardinalidade de B. Se $C(A,B)$ é igual a 1, então todas as soluções em B são dominadas por soluções da fronteira A. Por outro lado, se o valor retornado é igual a 0, isso aponta para uma situação oposta na qual nenhuma das soluções de B são dominadas por A. Para aplicação destas métricas, é necessário avaliar tanto $C(A,B)$ quanto $C(B,A)$, pois estes valores não são complementares. A Equação 10 apresenta como se obtém o valor da cobertura dadas duas fronteira Pareto aproximadas A e B (Zitzler, 1999).

$$C(A, B) = \frac{|\{y \in B : \exists x \in A, x \preceq y\}|}{|B|} \quad (10)$$

Neste trabalho, a métrica hipervolume é calculada considerando como ponto de referência (W) o maior valor de cada objetivo do problema, obtido após a execução de todas as replicações de todos os algoritmos. O valor do Hipervolume foi normalizado no intervalo (0, 1), independentemente, para cada problema. Para ambas as métricas, Hipervolume e Cobertura, os valores obtidos representam a diferença entre porcentagem, enquanto a métrica Cardinalidade fornece valores absolutos.

4.1 Planejamento estatístico

Foram empregados testes estatísticos a fim de detectar diferenças significativas entre os algoritmos e estimar sua magnitude, os quais foram realizados, independentemente, para cada métrica avaliada. Esta análise foi aplicada em um conjunto de dados composto pela avaliação das métricas dadas às soluções obtidas em 8 replicações de cada algoritmo em cada problema teste.

Para cada métrica, o experimento foi projetado como um planejamento completo em blocos aleatorizados (RCBD, *Randomized Complete Bloc Design*), sendo os algoritmos os níveis do fator experimental e os problemas como um fator de blocagem (Montgomery, 2008). Ao considerar os problemas como blocos, é possível modelar e remover os efeitos das diferentes instâncias nos resultados, e obter as diferenças reais no desempenho dos algoritmos, dados os múltiplos problemas teste utilizados. A hipótese nula de ausência de diferença entre os algoritmos foi testada contra a hipótese alternativa bilateral. Para evitar a necessidade de se atender às premissas de normalidade do teste F, foi utilizado o teste de Gore (Parsad, 2002), uma alternativa não-paramétrica robusta para análise de variância em blocos.

Após os testes de significância, os estimadores dos efeitos dos blocos (instâncias) foram obtidos (Montgomery, 2008) e subtraídos das amostras, permitindo, assim, uma estimativa problema-independente dos tamanhos dos efeitos de cada algoritmos. As magnitudes das diferenças de desempenho foram obtidas através do estimador de Hodges-Lehmann (HL) (Hodges & Lehmann, 1963), que fornece uma medida da mediana das diferenças entre duas amostras independentes. Uma descrição mais detalhada dos métodos estatísticos estão disponíveis em Batista et al. (2011).

4.2 Resultados e discussões

Os resultados obtidos nos testes aplicados mostram diferenças estatisticamente significativas entre os algoritmos avaliados. A média e o desvio padrão dos valores obtidos estão sumarizados na Tabela 1. A Tabela 2 sintetiza os resultados das análises estatísticas, mostrando a grandeza das diferenças estatisticamente significativas.

Para a métrica Hipervolume, todos os algoritmos apresentaram diferenças relativamente pequenas, como mostrado na Tabela 1, com o algoritmo P_ϵ apresentando um desempenho um pouco pior que os demais métodos. A análise estatística, apresentada na Tabela 2, confirma esta observação, mostrando que todos os algoritmos apresentaram diferenças relativamente pequenas, mas estatisticamente significantes, com o MOILS mostrando os melhores resultados (11% superior ao P_ϵ -ILS e 8% melhor que o NSGA-II). Estes resultados indicam que o MOILS, em geral, é capaz de obter melhores fronteiras por retornar um conjunto de soluções melhor distribuídas ou pontos mais próximos da fronteira Pareto-ótima.

As diferenças médias obtidas pela métrica Cardinalidade também foram relativamente modestas. A Tabela 1 mostra que, para cada problema, todos os algoritmos retornaram aproximadamente o mesmo número médio de soluções. A Tabela 2 mostra que, quando considerado todos os problemas, diferenças pequenas mas estatisticamente significantes foram observadas. Os resultados obtidos a partir do algoritmo MOILS apresentam um número maior de soluções presentes na fronteira de Pareto aproximada. Em média, o MOILS retorna 2.56 soluções a mais que o P_ϵ -ILS e 4.32 a mais que o NSGA-II.

Diferenças mais significativas nos resultados dos algoritmos podem ser observadas para a métrica de Cobertura. O MOILS supera a heurística P_ϵ -ILS por uma média de 0.7 e o NSGA-II por 0.32. Este é um indicador muito evidente de superioridade, uma vez ele indica que as fronteiras geradas pelos MOILS

Tabela 1. Média e desvio padrão (entre parênteses) dos valores obtidos pelas métricas avaliadas.

Instâncias (CMT)	Hipervolume			Cardinalidade		
	MOILS	P_{ϵ}	NSGA-II	MOILS	P_{ϵ}	NSGA-II
1X	0,91(0,16)	0,60(0,17)	0,86(0,16)	3,12(0,6)	6,25(3,6)	2,12(0,3)
1Y	0,95(0,02)	0,90(0,05)	0,87(0,07)	11,12(1,1)	8,62(1,5)	7,5(1,2)
2X	0,88(0,07)	0,77(0,09)	0,83(0,10)	8,12(2,8)	9,50(2,1)	4,00(1,6)
2Y	0,93(0,01)	0,82(0,05)	0,84(0,06)	33,62(6,5)	27,25(4,7)	31,12(5,3)
3X	0,94(0,05)	0,74(0,20)	0,91(0,05)	6,37(0,9)	8,87(3,5)	3,37(1,1)
3Y	0,96(0,00)	0,76(0,05)	0,86(0,04)	16,63(2,6)	13,63(2,9)	15,5(5,1)
4X	0,90(0,04)	0,83(0,08)	0,78(0,06)	15,25(2,6)	12,63(5,0)	5,0(2,5)
4Y	0,95(0,02)	0,88(0,06)	0,83(0,09)	42,13(3,3)	32,63(5,5)	35,5(8,0)
5X	0,91(0,04)	0,82(0,05)	0,85(0,06)	17,25(4,2)	16,25(4,3)	8,25(5,2)
5Y	0,94(0,01)	0,82(0,04)	0,83(0,03)	51,63(5,9)	52,0(10,6)	59,88(14,1)
11X	0,94(0,03)	0,87(0,02)	0,87(0,03)	44,75(5,5)	42,13(5,3)	40,75(7,1)
11Y	0,91(0,08)	0,65(0,08)	0,70(0,03)	27,63(5,6)	20,25(3,8)	20,5(2,8)
12X	0,96(0,02)	0,75(0,04)	0,87(0,08)	14,63(3,7)	9,25(2,2)	14,38(3,4)
12Y	0,98(0,01)	0,85(0,06)	0,91(0,03)	32,75(3,7)	14,50(3,6)	20,00(5,4)
Média	0,93(0,04)	0,79(0,07)	0,84(0,06)	23,21(3,5)	19,55(4,19)	19,13(4,51)

Instâncias (CMT)	Cobertura		
	MOILS	P_{ϵ}	NSGA-II
1X	0,80(0,29)	0,53(0,09)	0,65(0,15)
1Y	0,53(0,24)	0,63(0,20)	0,36(0,12)
2X	0,63(0,18)	0,54(0,13)	0,65(0,30)
2Y	0,67(0,06)	0,42(0,09)	0,55(0,15)
3X	0,59(0,37)	0,60(0,14)	0,72(0,25)
3Y	0,74(0,08)	0,34(0,05)	0,56(0,13)
4X	0,67(0,17)	0,59(0,10)	0,48(0,09)
4Y	0,68(0,12)	0,52(0,16)	0,40(0,12)
5X	0,64(0,19)	0,59(0,15)	0,56(0,18)
5Y	0,73(0,09)	0,39(0,06)	0,55(0,08)
11X	0,72(0,08)	0,56(0,06)	0,39(0,10)
11Y	0,71(0,14)	0,55(0,06)	0,41(0,10)
12X	0,73(0,11)	0,42(0,08)	0,58(0,07)
12Y	0,80(0,10)	0,31(0,02)	0,49(0,09)
Média	0,69(0,16)	0,50(0,10)	0,53(0,14)

Tabela 2. Diferenças estimadas no desempenho médio entre os algoritmos (linha $\times$ coluna) para as métricas consideradas. Somente os efeitos estatisticamente significativos a um nível de confiança de 95% (ajustado para o teste de múltiplas hipóteses usando Correção Dunn-Sidak (Dunn, 1961)) são apresentados. Os valores positivos indicam um valor médio maior para o algoritmo da linha. Informações redundantes foram removidas da tabela a fim de melhorar a legibilidade.

	Hipervolume		Cobertura		Cardinalidade	
	P_{ϵ} -ILS	NSGA-II	P_{ϵ} -ILS	NSGA-II	P_{ϵ} -ILS	NSGA-II
MOILS	0,11	0,08	0,70	0,32	2,56	4,32
P_{ϵ} -ILS	-	-0,03	-	-0,28	-	n.s

*“n.s.” significa “estatisticamente não significativo”.

foram capazes de dominar, em média, cerca de 70% das fronteiras correspondentes obtidas pelo P_{ϵ} -ILS, e 32% no caso do NSGA-II. Para simplificar a apresentação dos valores relativos de cobertura, uma versão generalizada da métrica Cobertura de Dois Conjuntos, denominada Cobertura de Vários Conjuntos (Batista et al., 2012), foi utilizado para gerar os dados apresentados na Tabela 1. Ao invés de quantificar o quanto um

determinado algoritmo abrange outra, essa métrica generalizada calcula a quantidade que um determinado algoritmo cobre a união das fronteiras finais retornados por todos os algoritmos, exceto em a fronteira obtida por ele mesmo.

A Figura 13 mostra dois exemplos de fronteiras obtidas pelos três algoritmos. Uma vez que a fronteira Pareto-ótima é desconhecida para o problema considerado, não é possível avaliar a quantidade absoluta da fronteira retornada pelos algoritmos propostos neste capítulo. Contudo, um ponto extremo da fronteira é equivalente à instância correspondente do VRPSPD. A Tabela 3 apresenta uma comparação deste ponto extremo com os melhores resultados conhecidos para o VRPSPD (Subramanian et al., 2010). Pode-se perceber que, neste ponto, o MOILS encontra soluções muito próximas da melhor solução conhecida da literatura.

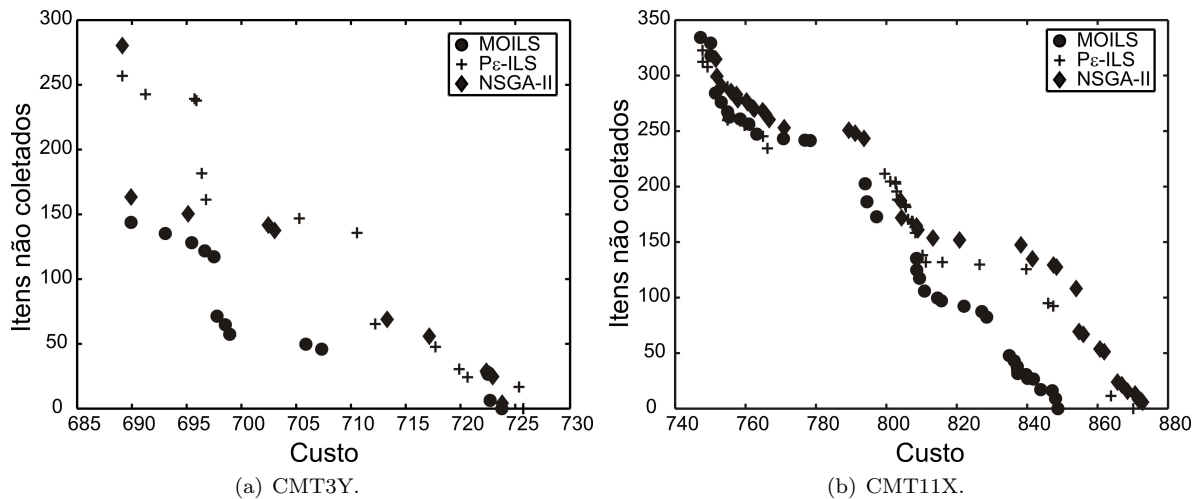


Figura 13. Exemplo das fronteiras obtidas pelos algoritmos implementados.

Tabela 3. Comparação do MOILS com os melhores resultados conhecidos para o VRPSPD mono-objetivo (100% das coletas atendidas).

	Instâncias (CMT)						
	1X	1Y	2X	2Y	3X	3Y	12X
MOILS	466,77	466,77	688,05	693,09	721,40	723,28	665,85
Melhor	466,77 ²	466,77 ²	668,77 ¹	663,25 ¹	721,27 ²	721,27 ²	644,70 ¹
Erro*(%)	0,00	0,00	2,88	4,50	0,02	0,28	3,28

	Instâncias (CMT)						
	12Y	11X	11Y	4X	4Y	5X	5Y
MOILS	664,00	848,72	850,67	857,44	855,74	1037,25	1032,68
Melhor	659,52 ¹	833,92 ¹	830,39 ¹	852,46 ¹	852,35 ¹	1029,25 ¹	1029,25 ¹
Erro**(%)	0,68	1,77	2,44	0,58	0,40	0,78	0,33

¹ Resultados reportados em Subramanian et al. (2010).

² Solução ótima (Subramanian et al., 2011).

** Diferença percentual entre MOILS e os melhores conhecidos.

5. Conclusões

Diversos problemas reais de roteamento de veículos com serviços de coleta e entrega possuem uma natureza multiobjetivo, mas, apesar disso, esta abordagem é ainda pouco explorada na literatura. Este capítulo apresentou uma formulação matemática adaptada ao problema de roteamento de veículos multiobjetivo denominado problema de roteamento de veículos multiobjetivo com coleta seletiva.

Devido à complexidade computacional do problema tratado, foi apresentado um conjunto de métodos heurísticos para solucioná-lo. Assim, explorou-se as especificidades das meta-heurísticas baseadas em

busca local, em população e nos métodos tradicionais de resolução de problemas multiobjetivo, resultando, respectivamente, nos algoritmos: MOILS, NSGA-II e P_ϵ -ILS

Após análises estatísticas, os testes de hipótese mostram diferenças significativas no desempenho dos algoritmos, apontando uma superioridade do MOILS em relação aos outros algoritmos. O bom desempenho do algoritmo baseado no MOILS pôde ser também observado na comparação com os resultados da literatura. Neste trabalho, um dos pontos extremos da fronteira, no qual todas as demandas de coleta são atendidas, é comparado com os melhores resultados conhecidos do problema de roteamento de veículos com coleta e entrega simultânea. Os resultados mostraram que as soluções obtidas pelo MOILS se aproximam das melhores da literatura com um erro relativo médio de 0,9%, sendo que, em muitas instâncias, este erro é quase nulo, confirmando o bom desempenho deste algoritmo.

6. Agradecimentos

Os autores agradecem ao Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) pelo apoio financeiro (edital 004/2010/PIBIT-CNPq-UFVJM, projeto PQ:306910/2006-3 e 472446/2010-0) e à Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) pela concessão de bolsa de estudos por meio do Programa de Fomento à Pós-Graduação (PROF).

Referências

- Batista, L.S.; Campelo, F.; Guimarães, & F. G. Ramírez, J.A., Pareto cone ϵ -dominance: Improving convergence and diversity in multiobjective evolutionary algorithms. In: *Proceedings of the 6th international conference on Evolutionary Multi-criterion Optimization*. Heidelberg, Germany: Springer-Verlag, v. 6576 de *Lecture Notes in Computer Science*, p. 76–90, 2011.
- Batista, L.S.; Campelo, F.; Guimarães, & F. G. Ramírez, J.A., The cone ϵ -dominance: An approach for evolutionary multiobjective optimization evolutionary computation. *Evolutionary Computation*, 2012 [Submitted].
- Chankong, V. & Haimes, Y.Y., *Multiobjective Decision Making: Theory and Methodology*. New York, USA: Elsevier Science, 1983.
- Coello Coello, C.A.; Lamont, G.B. & van Veldhuizen, D.A., *Evolutionary Algorithms for Solving Multi-Objective Problems*. 2a edição. New York, USA: Springer Science, 2007.
- Deb, K.; Pratap, A.; Agarwal, S. & Meyarivan, T., A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- Dunn, O., Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, 1961.
- Geiger, M.J., Foundations of the pareto iterated local search metaheuristic. In: *Proceedings of the 18th International Conference on Multiple Criteria Decision Making*. Chania, Greece, p. 19–23, 2006.
- Gribovskaia, I.; Halskau, O.; Laporte, G. & Vlcek, M., General solutions to the single vehicle routing problem with pickups and deliveries. *European Journal of Operational Research*, 180(2):568 – 584, 2007.
- Gutiérrez-Jarpa, G.; Desaulniers, G.; Laporte, G. & Marianov, V., A branch-and-price algorithm for the vehicle routing problem with deliveries, selective pickups and time windows. *European Journal of Operational Research*, 206(2):341 – 349, 2010.
- Hodges, J.L. & Lehmann, E.L., Estimation of location based on ranks. *Annals of Mathematical Statistics*, 34:598–611, 1963.
- Holland, J.H., *Adaptation in Natural and Artificial Systems*. Cambridge, USA: MIT Press, 1992.
- Jozefowicz, N.; Semet, F. & Talbi, E.G., From single-objective to multi-objective vehicle routing problems: Motivations, case studies, and methods. In: Golden, B.L.; Raghavan, S. & Wazil, E.A. (Eds.), *The Vehicle Routing Problem: Latest Advances and New Challenges*. Berlin, Germany: Springer, v. 43 de *Operations Research/Computer Science Interfaces Series*, p. 445–471, 2008.
- Lourenço, H.R.; Martin, O.C. & Stützle, T., Iterated local search. *AIROnews*, VIII(3):11–14, 2003.
- Montané, F.A.T. & Galvão, R.D., A tabu search algorithm for the vehicle routing problem with simultaneous pick-up and delivery service. *Computers and Operations Research*, 33(3):595–619, 2006.
- Montgomery, D.C., *Design and Analysis of Experiments*. 7a edição. New York, USA: J. Wiley & Sons, 2008.
- Parsad, R., Non-parametric methods in analysis of experimental data. In: R. Parsad R. Srivastava, V.K.G. (Ed.), *Design and Analysis of Agricultural Experiments*. New Delhi, India: Indian Agricultural Statistics Research Institute, Mod. V, p. 693–694, 2002.
- Penna, P.H.V.; Subramanian, A. & Ochi, L.S., An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. *Journal of Heuristics*, :1–322012H<http://dx.doi.org/10.1007/s10732-011-9186-y>.
- Prins, C., A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12):1985–2002, 2004.
- Salhi, S. & Nagy, G., A cluster insertion heuristic for single and multiple depot vehicle routing problems with backhauling. *Journal of Operational Research Society*, 50(10):1034–1042, 1999.

- Subramanian, A.; Drummond, L.M.A.; Bentes, C.; Ochi, L.S. & Farias, R., A parallel heuristic for the vehicle routing problem with simultaneous pickup and delivery. *Computers & Operations Research*, 37(11):1899 – 1911, 2010.
- Subramanian, A.; Uchoa, E.; Pessoa, A.A. & Ochi, L.S., Branch-and-cut with lazy separation for the vehicle routing problem with simultaneous pickup and delivery. *Operations Research Letters*, 39(5):338–341, 2011.
- Sural, H. & Bookbinder, J.H., The single-vehicle routing problem with unrestricted backhauls. *Networks*, 41(3):127–136, 2003.
- Zitzler, E., Evolutionary Algorithm for Multiobjective Optimization: Methods and Applications. PhD Thesis, Swiss Federal Institute of Technology Zurich, Computer Engineering and Networks Laboratory, Zurich, Switzerland, 1999.

Notas Biográficas

Luciana Pereira de Assis é graduada em Ciência da Computação (Centro Universitário de Belo Horizonte, 2004) e mestre em Ciência da Computação (UFMG, 2007). Atualmente é professora Assistente do departamento de Computação da UFVJM e doutoranda junto ao Programa de Pós-Graduação em Engenharia Elétrica da UFMG.

André Luiz Maravilha Silva é graduado em Sistemas de Informação (UFVJM, 2011) e atualmente é mestrando junto ao Programa de Pós-Graduação em Engenharia Elétrica da UFMG.

Felipe Campelo Franca Pinto é graduado em Engenharia Elétrica (UFMG, 2003), mestre e doutor em Engenharia (Hokkaido University, Japão, 2006 e 2009, respectivamente). Atualmente é professor Adjunto do departamento de Engenharia Elétrica da UFMG, atuando também como co-investigador junto ao Laboratory of Hybrid Systems, Hokkaido University.

Alessandro Vivas Andrade é graduado, mestre e doutor em Engenharia Elétrica (UFMG, 1999, 2002 e 2008, respectivamente). Atualmente é professor Adjunto do Departamento de Computação da UFVJM.

Jaime Arturo Ramírez é graduado e mestre em Engenharia Elétrica (UFMG, 1986 e 1990, respectivamente) e doutor em Engenharia Elétrica (Imperial College London, UK, 1994). Atualmente é professor Titular do Departamento de Engenharia Elétrica da UFMG.